

Specification, version 1

1. Connection

- A client sends all requests of one run on it and MUST NOT open a second; if it closes early, the client reports an error.
- A client SHOULD wait for each response, but a server MUST accept pipelined requests and answer them in order.
- A server MAY refuse a connection by closing it before reading anything. Otherwise it closes only after a connection error (§6), a body it cannot finish (§4), or a time limit (idle, slow request, or a client not taking frames). Each limit SHOULD be at least 10 s; unknown frames are not activity.
- Unless a body fails or the client stalls, a server never closes with a received request unanswered, so a request with no response was not processed.

0																1																2																3															
0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9																								
Length (24)																Type (8)																																															
Flags (8)																Stream ID (24)																																															

- | tag | name | tag | name | tag | name | tag | name | tag | name |
|-----|---------|-----|---------|-----|------------|-----|--------|-----|----------------|
| 1 | :method | 3 | :status | 5 | user-agent | 7 | server | 9 | content-type |
| 2 | :path | 4 | host | 6 | accept | 8 | date | 10 | content-length |

- Literal names are lowercase HTTP token characters (RFC 9110 §5.6.2). `:` is not one, so pseudo-headers are sent by tag only. Senders MUST use the tag for a table name; receivers also accept the other table names as literals.
- Values may hold any byte except NUL, CR and LF. `content-length` is 1 to 18 digits, read as a number. Senders SHOULD use the shortest len; receivers accept both forms.
- Pseudo-headers come first. `:method`, `:path`, `:status`, `host` and `content-length` appear at most once.
- A request has one `:method` (a case-sensitive token), one `:path` (starting with `/`, at most 1024 bytes), no `:status`, and SHOULD have `host`.
- A response has one `:status` from 200 to 599 and no `:method` or `:path`.
- Breaking any rule above other than a SHOULD or the senders' duty to use tags, using tags 11 to 255, or running past the payload makes a block malformed.

6. Server

The server cuts `:path` at the first `?` or `#`, decodes `%XX` escapes and looks the path up under its document root; a directory, or a path ending in `/`, means its `index.html`. Every response carries `content-type`, `content-length`, `server` and `date`. A found file gets 200 and the file as DATA, an error gets a short text body, and HEAD gets the headers only. The first matching row decides:

status	when
400	the header block is malformed or over 65 535 bytes, or the path has a bad <code>%</code> escape or <code>%00</code>
405	the method is well-formed but not GET or HEAD
403	the path would lead outside the root, or the file may not be read
404	there is no regular file to serve
500	opening or reading the file failed in some other way

Treatment of symbolic links and dotfiles is left to the server.

These are stream errors: the response goes on the request's stream (after any body is discarded) and the connection stays open. Connection errors, which win over stream errors, mean the frame sequence can no longer be trusted:

- HEADERS on stream 0, or with an ID not higher than every earlier one;
- DATA with no request open, or on another stream;
- a new HEADERS before the open request's `END_STREAM`.

The server answers requests completed before the bad frame, sends a 400 response on stream 0 whose body gives the reason, and closes. On EOF inside a frame or request, it just closes.

7. Client

The client matches responses to its requests in order. A HEADERS frame on stream 0, at any point, starts a connection-error message: the client reads it, reports the reason and stops. Otherwise it closes on a protocol error:

- a HEADERS or DATA frame for any stream but the oldest unanswered request;
- DATA before HEADERS or after `END_STREAM`, or a second HEADERS in one response;
- DATA in a response to HEAD, unless the status is 400;
- a malformed or oversized response block, or a `content-length` mismatch;
- EOF in the middle of a response, or a read timeout.

8. Design notes

HTTP/2's frame header is 9 bytes: Length 24, Type 8, Flags 8, a reserved bit and a 31-bit Stream ID. Its drafts used 8 bytes, like BHTTP/1, until draft 14 (2014) widened Length from 14 to 24 bits so a receiver could opt in to larger frames. The default stayed 2^{14} so one large frame cannot hold up other streams. Flags are defined per frame type, so 8 bits suffice. The Stream ID is a 32-bit word minus the reserved bit, split between client (odd) and server (even). BHTTP/1 uses 24 / 8 / 8 / 24:

- Length 24. v1's own sizes fit in 16 bits; the third byte allows larger frames later. The width can never change, because §3's skipping depends on every version reading Length the same way.
- Type 8 and Flags 8. Whole bytes need no masking, and the four fields fill two 32-bit words exactly. New frame types are safe for v1 peers (§3); a new flag or tag is not, so a v2 negotiates those first.
- Stream ID 24. v1 answers in order and the server never opens streams, so there is no odd/even split. The ID matches responses to requests and exposes stale frames. At the 19 000 requests a second bserve manages on loopback, 2^{24} IDs last about 15 minutes.

Like HPACK, BHTTP/1 indexes common names in one byte and sends the rest as length-prefixed literals, without HPACK's dynamic table, Huffman coding or indexed values. With no preface, a stray HTTP/1.1 GET `/` reads as an unknown frame (Length 0x474554) and is skipped until the idle limit. For v2, a client sends a new HELLO frame type with a plain-v1 first request; a v2 server answers it before responding, a v1 server skips it.